

球体-AABB 碰撞检测：两阶段廉价过滤算法

本文档总结了您提出的用于球体 (Sphere) 与 AABB (轴对齐包围盒) 碰撞检测的高性能算法。

1. 算法目标

传统算法 (如 `SqDistPointAABB`) 通常会计算球心到 AABB 最近点的距离平方 (d^2)，这不可避免地需要执行多次高功耗的乘法 (FMUL) 运算。

本算法的目标是**重构计算过程**，引入一个**“廉价过滤器”阶段**。此阶段只使用低功耗的减法 (FSUB) 和比较 (FCMP) 操作，用于快速剔除 (Early-Out) 绝大多数明显不碰撞的情况，从而**避免执行任何乘法运算**。

只有当这个廉价过滤器无法证明“未碰撞”时 (即物体非常接近)，才启用包含乘法运算的“精确检测”阶段。

2. 核心思想：两阶段检测

算法分为两个阶段：

- 阶段 1 (廉价过滤器)：** 在每个轴上独立计算球心 C_s 超出 (Excess) AABB 边界的距离 `excess_axis`。然后，将这个一维的 `excess` 值与球体的完整半径 r_s 进行比较。如果任何一个轴的 `excess` 都大于 r_s ，则物体在几何上不可能碰撞。
- 阶段 2 (精确检测)：** 如果所有三个轴的廉价过滤器都失败了 (即在所有轴上 `excess_axis > r_s`)，我们就无法仅通过一维信息判断。此时，我们才授权执行高功耗的乘法运算，计算精确的距离平方 d^2 ，并与半径平方 r_s^2 进行最终比较。

3. 输入数据

- 球体 S ：** 中心 $C_s = (C_x, C_y, C_z)$ ，半径 r_s
- AABB B ：** 最小角点 $B_{min} = (min_x, min_y, min_z)$ ，最大角点 $B_{max} = (max_x, max_y, max_z)$

4. 详细算法流程

阶段 1: 廉价过滤器 (仅 减法/比较)

此阶段的目标是证明 `NO_COLLISION`。

1. 计算 X 轴超出距离 (`excess_x`)

- `excess_x = 0.0`
- `if (C_x < min_x) : (1 FCMP, 1 Branch)`
 - `excess_x = min_x - C_x (1 FSUB)`
- `else if (C_x > max_x) : (1 FCMP, 1 Branch)`
 - `excess_x = C_x - max_x (1 FSUB)`

2. 廉价过滤器 (轴 1)

- `** if (excess_x > r_s) : (1 FCMP, 1 Branch)`
 - `return NO_COLLISION (提前退出)**`

- 分析：此时已消耗 1 FSUB 和 3-4 次比较，**0 次乘法**。如果球体在 X 轴方向上离 AABB 足够远，算法在此处以最低功耗终止。

3. 计算 Y 轴超出距离 (excess_y)

- `excess_y = 0.0`
- ... (同 X 轴逻辑) ...

4. 廉价过滤器 (轴 2)

- `** if (excess_y > r_s) : (1 FCMP, 1 Branch)`
 - `return NO_COLLISION` (提前退出)**

5. 计算 Z 轴超出距离 (excess_z)

- `excess_z = 0.0`
- ... (同 X 轴逻辑) ...

6. 廉价过滤器 (轴 3)

- `** if (excess_z > r_s) : (1 FCMP, 1 Branch)`
 - `return NO_COLLISION` (提前退出)**

阶段 2：精确检测 (启动 乘法)

7. 执行高功耗计算

- 如果算法执行到这一步，意味着阶段 1 的所有廉价测试都失败了。
- 几何场景：物体非常接近，可能在 AABB 的“角”或“边”附近。
- 现在我们**必须**执行乘法来确认。
- 计算半径平方：
 - `r_s_sq = r_s \times r_s` (1 FMUL)
- 计算距离平方和：
 - `sq_dx = excess_x \times excess_x` (1 FMUL)
 - `sq_dy = excess_y \times excess_y` (1 FMUL)
 - `sq_dz = excess_z \times excess_z` (1 FMUL)
 - `d_sq = sq_dx + sq_dy + sq_dz` (2 FADD)

8. 最终比较

- `if (d_sq \le r_s_sq) : (1 FCMP, 1 Branch)`
 - `return COLLISION`
- `else :`
 - `return NO_COLLISION`

5. 性能分析与优势

此算法流程创建了一个级联（Cascade）的过滤器，将高功耗的计算（乘法）推迟到绝对必要时才执行。

最佳情况 (Best Case)

- 场景：远距离分离，例如在 X 轴上。

- **执行：** 算法在 **阶段 2** 提前退出。
- **成本：**
 - 乘法 (FMUL): 0
 - 减法 (FSUB): 1
 - 比较/分支 (FCMP/Branch): ≈ 3
- **优势：** 成功地以“零乘法”成本剔除了最常见的“未碰撞”情况。

最坏情况 (Worst Case)

- **场景：** 物体**正在碰撞**，或者在“角落附近”紧密接近但未碰撞。
- **执行：** 算法必须完整执行到 **阶段 8**。
- **成本：**
 - 乘法 (FMUL): 4 (1 次 for `r_s_sq`, 3 次 for `excess_sq`)
 - 减法/加法 (FSUB/FADD): ≈ 5
 - 比较/分支 (FCMP/Branch): ≈ 10
- **分析：** 即使在最坏情况下，此算法的成本也仅与标准算法（如 `sphere_vs_aabb_analysis_reconstructed.md` 中所述）的最坏情况相当。

6. 总结

您提出的算法是一种高效的混合策略。它在标准 `SqDistPointAABB` 算法之前，增加了一个“仅减法”的预过滤器。

- 它利用了这样一个事实：在大多数物理或游戏场景中，绝大多数物体对都处于**远距离分离**状态。
- 该算法能以极低的功耗（0 次乘法）处理这些“简单”情况。
- 它只为少数“困难”的（紧密接近或碰撞）情况保留了高功耗的乘法计算，实现了性能和功耗的优化平衡。